

Project Documentation

Import Data using Transform Maps (Spreadsheet)

Team ID: 6ab22227a614981d6d2c6451

Date: 25 September 2026

Institution: Alagappa Government Arts College, Karaikudi

Department: Bachelor of Computer Applications

Table of Contents

1. Introduction
2. Project Overview
3. Architecture
4. Setup Instructions
5. Folder Structure
6. Running the Application
7. API Documentation
8. Authentication
9. User Interface
10. Testing
11. Screenshots or Demo
12. Known Issues
13. Future Enhancements

1. Introduction

Project Title: Import Data using Transform Maps (Spreadsheet)

Team Members:

S.No	Name	Role	Responsibilities
1	VAIRAM KISHORE J	Project Lead & System Admin	Overall project management, ServiceNow instance configuration, ACL setup, Dashboard creation and administration.
2	SUDEEP KUMAR S	Data Engineer	Excel dataset preparation, Import Set configuration, Transform Map field mapping, and data validation.
3	VEERASEKARAN A	Testing & QA Lead	Functional testing, Coalesce validation testing, performance benchmarking, and bug tracking.
4	VELKUMAR S	Report & Dashboard Developer	Report creation (Pie Chart, Bar Chart, List View), Dashboard widget configuration, and UI documentation.

2. Project Overview

Purpose

The purpose of this project is to demonstrate and implement a robust, automated data migration pipeline within the ServiceNow platform. In modern enterprises, HR departments and system administrators frequently receive bulk employee data in the form of Excel spreadsheets that must be accurately imported into a centralized IT Service Management (ITSM) database. Traditional manual data entry is not only time-consuming but also highly error-prone, often resulting in duplicate records, inconsistent formatting, and data corruption.

This project addresses these challenges head-on by leveraging ServiceNow's native **Import Set** and **Transform Map** capabilities. The Import Set acts as a secure staging area where raw spreadsheet data is temporarily stored before being processed. The Transform Map then serves as the intelligent bridge between the staging table and the final target table, automatically mapping source fields to destination fields. Most critically, the project implements the **Coalesce** feature — a powerful deduplication mechanism that uses a designated unique field (Employee ID) to determine whether incoming records should be inserted as new entries or used to update existing ones.

Beyond data ingestion, the project extends into the domain of business intelligence by creating dynamic, real-time visual reports and consolidating them into a centralized

analytics dashboard. This ensures that stakeholders such as HR Managers and Department Heads can instantly access actionable insights regarding employee distribution across departments and geographical locations without needing to manually analyze raw data.

Features

The project encompasses the following key features and functionalities:

1. Bulk Data Import via Spreadsheet Upload The system accepts standard Excel (.xlsx) files through the ServiceNow "Load Data" module. Upon upload, the platform automatically parses the file contents and generates a corresponding Import Set staging table (u_employee_import), preserving the original column structure and data types.

2. Custom Target Table Creation A dedicated custom table (u_employee_test) was designed and configured within the ServiceNow platform to serve as the permanent repository for employee records. The table includes the following string-type fields:

- Employee ID (u_employee_id)
- Name (u_name)
- Email (u_email)
- Department (u_department)
- Location (u_location)

3. Automated Field Mapping with Transform Maps A Transform Map was configured to establish a logical relationship between the Import Set staging table and the custom target table. Using the Mapping Assist utility, each source column from the spreadsheet was mapped to its corresponding target field, ensuring seamless and accurate data transfer.

4. Intelligent Deduplication via Coalesce The Coalesce property was enabled on the Employee ID field within the Transform Map configuration. This ensures that when a spreadsheet is re-uploaded with records that share the same Employee ID as existing database entries, the system automatically updates those existing records with the new information rather than creating duplicate rows. This is the cornerstone of data integrity in the project.

5. Dynamic Reporting Engine Three distinct reports were generated from the target table data:

- **Employees by Department** — Rendered as a Pie Chart, providing a proportional breakdown of staff across organizational departments.

- **Employees by Location** — Rendered as a Bar Chart, offering a comparative view of employee distribution across geographical locations.
- **Employee List Report** — Rendered as a standard List View, displaying all employee records in a tabular format with sortable and filterable columns.

6. Centralized Analytics Dashboard All three reports were consolidated into a single, unified dashboard titled "Employee Analytics Dashboard." This dashboard serves as a one-stop analytical hub where managers can monitor workforce metrics in real time. To enable dashboard creation, the project involved modifying the Access Control List (ACL) for pa_dashboards by enabling the Admin Overrides configuration.

7. Data Validation and Integrity Assurance The project includes comprehensive validation mechanisms. After every transform operation, the Transform History log provides a detailed breakdown of how many records were "Inserted" (new entries) versus "Updated" (modified existing entries), along with any records that were "Ignored" due to errors. This transparency ensures complete auditability of the data migration process.

3. Architecture

The project follows a layered architecture pattern within the ServiceNow Platform-as-a-Service (PaaS) ecosystem. Each layer is responsible for a distinct phase of the data lifecycle, from ingestion to visualization.

Frontend (Presentation Layer)

The frontend of the application is entirely powered by the ServiceNow Standard User Interface (UI16) and Next Experience UI Framework. These are the native web-based interfaces provided by the ServiceNow platform, built using a combination of HTML5, CSS3, and JavaScript (AngularJS for UI16 / Web Components for Next Experience).

Key frontend components utilized in this project include:

- **Load Data Module UI:** A form-based interface that allows administrators to browse and upload Excel files from their local machine. The UI provides real-time feedback on the upload status, including file size, number of rows detected, and the auto-generated staging table name.
- **Form Views:** Standard ServiceNow form layouts used to configure Transform Maps, field mappings, and Coalesce settings. These forms are rendered dynamically based on the table schema and include client-side validation scripts.

- **List Views:** Tabular displays of records within both the Import Set table and the Target table. List views support inline editing, column sorting, filtering, and exporting to CSV/PDF.
- **Report Builder UI:** A drag-and-drop interface for creating visual reports. Users select the data source (table), choose a report type (Pie, Bar, List, etc.), configure grouping and aggregation parameters, and preview the output in real time.
- **Dashboard Canvas:** A grid-based layout engine where report widgets can be added, resized, and rearranged. The dashboard supports multiple tabs, auto-refresh intervals, and responsive scaling for different screen sizes.

Backend (Application Logic Layer)

The backend logic is entirely managed by the ServiceNow Application Server, which runs on a Java-based middleware layer (Apache Tomcat). The server-side scripting environment uses Mozilla Rhino, a JavaScript engine that executes server-side scripts within the ServiceNow platform.

Key backend components utilized in this project include:

- **Import Set API:** The core engine responsible for parsing uploaded Excel files, extracting row data, and populating the staging table. The API handles file type validation (accepting .xlsx, .csv, .xml), character encoding detection, and batch row insertion.
- **Transform Map Engine:** The orchestration engine that reads records from the staging table, applies field mapping rules, evaluates Coalesce conditions, and executes the appropriate database operation (INSERT or UPDATE) against the target table. The engine processes records sequentially and logs the outcome of each row in the Transform History.
- **GlideRecord API:** The primary server-side API for interacting with ServiceNow database tables. It provides methods for querying, inserting, updating, and deleting records. In this project, GlideRecord is used internally by the Transform Map Engine to check for existing records during Coalesce evaluation.
- **GlideAggregate API:** A specialized API for performing aggregate database queries (COUNT, SUM, AVG, MIN, MAX). The Report Builder uses GlideAggregate behind the scenes to calculate the data points displayed in Pie and Bar charts (e.g., counting employees grouped by Department).
- **ACL (Access Control List) Engine:** The security engine that evaluates every database operation against a set of configurable rules. In this project, the ACL rule for pa_dashboards was modified to enable "Admin Overrides," granting system administrators the ability to create and manage dashboards.

Database (Data Layer)

ServiceNow uses a relational database management system (RDBMS) as its underlying data store. Depending on the instance deployment, this is typically MariaDB (for on-premise or dedicated instances) or a managed relational database service (for cloud-hosted instances).

Key database components in this project include:

- **Import Set Table (u_employee_import):** A temporary staging table automatically generated by the Load Data module. This table mirrors the column structure of the uploaded Excel file and stores raw, unprocessed data. Import Set tables are designed to be transient — data can be cleaned up after successful transformation.
- **Custom Target Table (u_employee_test):** The permanent, production-grade table where validated employee records are stored. This table was manually created with a defined schema (5 string fields) and is the authoritative source of truth for all employee data within the system.
- **Transform History Table (sys_transform_history):** A system table that logs the results of every Transform Map execution, including timestamps, record counts (Inserted, Updated, Ignored, Error), and links to the source Import Set and target table.
- **Report Definition Table (sys_report):** A system table that stores the configuration metadata for every report created in the platform, including the data source, chart type, grouping parameters, and visual styling options.

4. Setup Instructions

Prerequisites

S.No	Prerequisite	Details
1	ServiceNow Developer Instance	A free Personal Developer Instance (PDI) must be provisioned from the ServiceNow Developer Portal (https://developer.servicenow.com/). The instance should be running on the latest release (e.g., Vancouver, Washington, or Xanadu).
2	Admin Access	The user must have the "admin" role assigned to their account on the ServiceNow instance. This is required for creating custom

		tables, configuring Transform Maps, modifying ACLs, and creating Dashboards.
3	Web Browser	A modern, standards-compliant web browser is required. Recommended: Google Chrome (latest version), Mozilla Firefox (latest version), or Microsoft Edge (Chromium-based).
4	Sample Dataset	An Excel spreadsheet (.xlsx format) containing structured employee data with columns: Employee ID, Name, Email, Department, Location. Minimum 5–10 sample rows recommended.
5	Stable Internet Connection	Since ServiceNow is a cloud-based SaaS platform, a stable and reliable internet connection is essential for accessing the developer instance and performing all configuration tasks.

Installation (Step-by-Step Setup Guide)

Since ServiceNow is a fully cloud-hosted Platform-as-a-Service (PaaS), there is no traditional software installation, dependency management, or environment variable configuration required. All setup is performed through the web-based ServiceNow UI.

Step 1: Provision a Developer Instance

1. Navigate to <https://developer.servicenow.com/>.
2. Create a free developer account (or log in if you already have one).
3. Click on "Start Building" and request a Personal Developer Instance (PDI).
4. Wait for the instance to be provisioned (typically 2–5 minutes).
5. Note down your instance URL (e.g., <https://dev270907.service-now.com>).
6. Log in using the admin credentials provided.

Step 2: Prepare the Sample Excel Dataset

1. Open Microsoft Excel or Google Sheets.
2. Create a new spreadsheet with the following column headers in Row 1:
 - employee_id
 - name
 - email
 - department
 - location

3. Populate at least 5–10 rows of sample employee data.
4. Save the file as Employee_Data.xlsx in .xlsx format.

Step 3: Create the Custom Target Table

1. In your ServiceNow instance, navigate to System Definition > Tables.
2. Click "New" to create a new table.
3. Set the table label as "Employee Test" (the system name will auto-generate as u_employee_test).
4. Add the following columns (all of type "String"):
 - Employee ID → u_employee_id
 - Name → u_name
 - Email → u_email
 - Department → u_department
 - Location → u_location
5. Click "Submit" to save the table.

Step 4: Upload Data via Load Data Module

1. Navigate to System Import Sets > Load Data.
2. In the "Import Set Table" field, select "Create table."
3. Name the new table u_employee_import.
4. Click "Choose File" and select your Employee_Data.xlsx file.
5. Click "Upload" and wait for the confirmation message.

Step 5: Create and Configure the Transform Map

1. Navigate to System Import Sets > Transform Maps.
2. Click "New" to create a new Transform Map.
3. Set the Source table to u_employee_import.
4. Set the Target table to u_employee_test.
5. Click "Submit."
6. In the Field Maps related list, click "Mapping Assist."
7. Map each source field to its corresponding target field.

8. Locate the Employee ID field mapping and set Coalesce to True.
9. Save the configuration.

Step 6: Execute the Transform

1. Navigate to System Import Sets > Import Sets.
2. Open the Import Set that was created during the upload.
3. Click "Transform" and select the Transform Map you configured.
4. Monitor the Transform History for the results (Inserted / Updated / Ignored counts).

Step 7: Configure Reports

1. Navigate to Reports > Create New.
2. Create three reports:
 - Report 1: Source = u_employee_test, Type = Pie Chart, Group By = Department.
 - Report 2: Source = u_employee_test, Type = Bar Chart, Group By = Location.
 - Report 3: Source = u_employee_test, Type = List, Columns = All fields.

Step 8: Configure the Dashboard

1. Navigate to System Security > Access Control (ACL).
2. Search for the ACL rule on pa_dashboards.
3. Enable "Admin Overrides" and save.
4. Navigate to Performance Analytics > Dashboards.
5. Click "New" and create a dashboard titled "Employee Analytics Dashboard."
6. Add the three reports as widgets to the dashboard canvas.

5. Folder Structure

Since this project is entirely built and hosted within the ServiceNow cloud platform (PaaS), there is no traditional file system or folder hierarchy like a Node.js or React application. Instead, the project's logical structure is organized as a set of ServiceNow platform artifacts — tables, configurations, and UI components that reside within the instance database.

Below is the logical organization of all project components:

ServiceNow Instance (<https://dev270907.service-now.com>)



- | | └─ Field Map: department → u_department
- | | └─ Field Map: location → u_location
- | |
- | └─ Transform History # Execution logs (Inserted/Updated/Ignored)
- |
- └─ 📁 Reports (Visualization Layer)
 - | └─ Employees by Department # Pie Chart Report
 - | └─ Employees by Location # Bar Chart Report
 - | └─ Employee List Report # List View Report
 - |
 - └─ 📁 Dashboards (Presentation Layer)
 - | └─ Employee Analytics Dashboard # Centralized Dashboard
 - | └─ Widget 1: Pie Chart (Department)
 - | └─ Widget 2: Bar Chart (Location)
 - | └─ Widget 3: List View (All Employees)
 - |
 - └─ 📁 Security (Access Control Layer)
 - | └─ ACL: pa_dashboards # Modified: Admin Overrides = True
 - |
 - └─ 📁 External Files (Local Machine)
 - | └─ Employee_Data.xlsx # Original Excel file (Initial Upload)
 - | └─ Employee_Data_v2.xlsx # Updated Excel file (Coalesce Test)

6. Running the Application

Since ServiceNow is a cloud-based SaaS/PaaS platform, there are no local servers to start, no terminal commands to execute, and no build processes to run. The application is always "running" on the ServiceNow cloud infrastructure and is accessed entirely through a web browser.

Accessing the Application

1. Open your web browser (Google Chrome recommended).
2. Navigate to your ServiceNow instance URL:

<https://dev270907.service-now.com>

(Replace dev270907 with your actual instance ID)

3. Log in with your administrator credentials:
 - Username: admin
 - Password: [Your Instance Password]

Key Navigation Paths

Once logged in, use the Application Navigator (left sidebar) to access the following modules:

Module Path	Purpose
System Import Sets > Load Data	Upload a new Excel spreadsheet and create an Import Set.
System Import Sets > Transform Maps	View, create, or modify Transform Map configurations.
System Import Sets > Import Sets	View all Import Set records and trigger transforms.
u_employee_test.list (URL suffix)	Directly view all records in the custom Employee Test table.
Reports > View / Run	Access and run existing reports.
Reports > Create New	Create a new report from scratch.
Performance Analytics > Dashboards	View and manage custom dashboards.
System Security > Access Control (ACL)	Manage security rules and ACL configurations.

Running a Data Import (End-to-End Workflow)

1. Navigate to System Import Sets > Load Data.
2. Select "Create table" or choose the existing u_employee_import table.
3. Click "Choose File" and select your .xlsx file.

4. Click Upload. Wait for the success confirmation.
5. Navigate to System Import Sets > Import Sets.
6. Open the newly created Import Set record.
7. Click Transform.
8. Select the "Employee Transform Map" from the dropdown.
9. Click Transform to execute.
10. Review the Transform History for results.
11. Navigate to u_employee_test.list to verify the imported records.
12. Open the Employee Analytics Dashboard to view updated visualizations.

7. API Documentation

ServiceNow provides a comprehensive set of built-in APIs that power the data operations in this project. While no custom REST APIs were developed for this project, the following native ServiceNow APIs are utilized internally by the platform modules:

7.1 Import Set API

The Import Set API is used internally by the "Load Data" module to process uploaded files and populate staging tables.

Endpoint:

POST /api/now/import/{staging_table_name}

Method: POST

Request Headers:

Header	Value
Content-Type	application/json
Accept	application/json
Authorization	Basic {base64_encoded_credentials}

Request Body (Example):

json

{

```
"u_employee_id": "EMP001",  
"u_name": "Vairam Kishore",  
"u_email": "vairam@example.com",  
"u_department": "IT",  
"u_location": "Chennai"  
}
```

Response (Example):

```
json  
{  
  "result": {  
    "transform_map": "Employee Transform Map",  
    "table": "u_employee_import",  
    "display_name": "label",  
    "display_value": "EMP001",  
    "record_link": "https://dev270907.service-  
now.com/api/now/table/u_employee_import/abc123",  
    "status": "inserted",  
    "status_message": "Record inserted successfully"  
  }  
}
```

7.2 Table API (GlideRecord)

The Table API provides RESTful access to ServiceNow tables. It is the programmatic equivalent of the GlideRecord server-side API.

Retrieve All Employee Records:

GET /api/now/table/u_employee_test

Query Parameters:

Parameter	Description	Example
-----------	-------------	---------

sysparm_query	Encoded query string to filter results	u_department=IT
sysparm_limit	Maximum number of records to return	10
sysparm_fields	Comma-separated list of fields to include	u_name,u_email,u_department
sysparm_display_value	Return display values instead of sys_ids	true

Response (Example):

```

json
{
  "result": [
    {
      "u_employee_id": "EMP001",
      "u_name": "Vairam Kishore",
      "u_email": "vairam@example.com",
      "u_department": "IT",
      "u_location": "Chennai"
    },
    {
      "u_employee_id": "EMP002",
      "u_name": "Sudeep Kumar",
      "u_email": "sudeep@example.com",
      "u_department": "HR",
      "u_location": "Bangalore"
    }
  ]
}

```

7.3 Aggregate API (GlideAggregate)

The Aggregate API is used by the Report Builder to compute statistical summaries.

Count Employees by Department:

GET

/api/now/stats/u_employee_test?sysparm_count=true&sysparm_group_by=u_department

Response (Example):

json

```
{
  "result": [
    { "groupby_fields": [{"field": "u_department", "value": "IT"}], "stats": {"count": "5"} },
    { "groupby_fields": [{"field": "u_department", "value": "HR"}], "stats": {"count": "3"} },
    { "groupby_fields": [{"field": "u_department", "value": "Finance"}], "stats": {"count": "2"} }
  ]
}
```

8. Authentication

Authentication Mechanism

ServiceNow implements a multi-layered authentication and authorization framework to secure all platform interactions. The project leverages the following authentication mechanisms:

Session-Based Authentication (Web UI): When a user logs into the ServiceNow instance through the web browser, the platform authenticates the credentials against its internal user directory (or an integrated LDAP/SSO provider). Upon successful authentication, the server generates a unique session token (`glide_session_token`) that is stored as an HTTP cookie in the user's browser. This session token is automatically included in every subsequent HTTP request, allowing the server to identify and authorize the user without requiring repeated login.

Basic Authentication (REST API): For programmatic access via REST APIs (as documented in Section 7), ServiceNow supports HTTP Basic Authentication. The client encodes the username and password in Base64 format and includes it in the Authorization header of each request. While Basic Auth is suitable for development and testing environments, OAuth 2.0 is recommended for production integrations.

Authorization (Role-Based Access Control)

ServiceNow uses a Role-Based Access Control (RBAC) model to govern what actions a user can perform within the platform. The following roles are relevant to this project:

Role	Description	Permissions in this Project
admin	Full system administrator	Can create tables, configure Transform Maps, modify ACLs, create dashboards, and access all modules.
import_admin	Import Set administrator	Can upload files, create Import Sets, and execute Transform Maps.
report_admin	Report administrator	Can create, modify, and share reports.
pa_admin	Performance Analytics admin	Can create and manage PA dashboards and widgets.
itil	Standard ITSM user	Can view records and run existing reports (read-only access to custom tables).

ACL Configuration for Dashboards

A critical security configuration in this project involved modifying the ACL rule for the `pa_dashboards` table. By default, only users with the `pa_admin` role can create new dashboards. To enable the system administrator to create dashboards without explicitly assigning the `pa_admin` role, the following modification was made:

1. Navigated to System Security > Access Control (ACL).
2. Searched for the ACL rule governing `pa_dashboards` with the operation "create."
3. Checked the "Admin Overrides" checkbox.
4. Saved the ACL record.

This configuration ensures that any user with the `admin` role automatically inherits the ability to create and manage PA dashboards, bypassing the need for the specific `pa_admin` role assignment.

9. User Interface

The project's user interface is built entirely using ServiceNow's native UI components. Below is a detailed description of each major UI screen and its features:

9.1 Load Data Screen

- **Purpose:** Allows the administrator to upload an Excel spreadsheet into the platform.
- **Key UI Elements:**
 - File browser button ("Choose File") for selecting the .xlsx file from the local machine.
 - Dropdown menu for selecting an existing Import Set table or creating a new one.
 - "Upload" button that initiates the file parsing and staging process.
 - Progress indicator showing the upload status.
 - Confirmation message displaying the number of rows imported and the staging table name.

9.2 Transform Map Configuration Screen

- **Purpose:** Allows the administrator to define the relationship between the staging table and the target table.
- **Key UI Elements:**
 - Source table selector (dropdown).
 - Target table selector (dropdown).
 - "Mapping Assist" button that opens an interactive field-mapping wizard.
 - Field Maps related list showing all configured mappings with Coalesce checkboxes.
 - "Auto Map Matching Fields" button for automatic field detection based on name similarity.

9.3 Target Table List View

- **Purpose:** Displays all employee records stored in the u_employee_test table.
- **Key UI Elements:**
 - Tabular grid showing all fields (Employee ID, Name, Email, Department, Location).
 - Column headers that support click-to-sort functionality (ascending/descending).
 - Filter bar for applying conditions (e.g., "Department = IT").

- Pagination controls for navigating through large datasets.
- "Export" button for downloading data as CSV, Excel, or PDF.

9.4 Report Views

Pie Chart (Employees by Department):

- A circular chart divided into colored segments, each representing a department.
- Hover tooltips showing the exact count and percentage for each segment.
- Legend panel listing all department names with their corresponding colors.

Bar Chart (Employees by Location):

- A horizontal/vertical bar chart with each bar representing a geographical location.
- Axis labels showing location names and employee counts.
- Hover tooltips displaying exact values.

List View Report:

- A formatted tabular report displaying all employee records.
- Supports grouping, sorting, and conditional highlighting.

9.5 Employee Analytics Dashboard

- **Purpose:** A centralized, single-screen view consolidating all three reports.
- **Key UI Elements:**
 - Dashboard title bar ("Employee Analytics Dashboard").
 - Three widget panels arranged in a responsive grid layout.
 - Each widget displays one report (Pie, Bar, or List).
 - Auto-refresh capability to reflect real-time data changes.
 - "Edit" mode for rearranging, resizing, or removing widgets.

10. Testing

Testing Strategy

The project employed a comprehensive testing strategy covering functional correctness, data integrity, performance benchmarks, and user acceptance criteria.

Testing was conducted manually through the ServiceNow web interface, with results documented in structured test case tables.

Testing Tools

Tool	Purpose
ServiceNow Transform History	Validates the outcome of each transform execution (Inserted/Updated/Ignored counts).
ServiceNow List Views	Used to manually verify that records in the target table match the expected data from the spreadsheet.
Browser Developer Tools (F12)	Used to monitor page load times, network requests, and JavaScript errors during dashboard rendering.
Manual Comparison	Side-by-side comparison of the original Excel file with the target table records to verify data accuracy.

Functional Test Cases

Test Case ID	Scenario	Test Steps	Expected Result	Actual Result	Pass/Fail
FT-01	File Upload Validation	Upload a valid .xlsx file via Load Data.	System accepts the file and creates the Import Set table.	File processed successfully. Staging table created.	Pass
FT-02	Invalid File Rejection	Attempt to upload a .txt or .doc file.	System rejects the file with an error message.	Error displayed: "Unsupported file format."	Pass
FT-03	Transform Map Execution	Run the Transform Map on the staged data.	All rows are inserted into the target table.	All rows inserted. Transform History shows "Complete."	Pass
FT-04	Field Mapping Accuracy	Compare source fields with target fields after transform.	Each source column maps to the correct target column.	All 5 fields mapped correctly. No mismatches.	Pass

FT-05	Coalesce Deduplication	Re-upload the same Excel file and run transform again.	System updates existing records instead of duplicating.	Transform History shows "Updated" for all rows. Zero new inserts.	Pass
FT-06	Coalesce with Mixed Data	Upload a file with both existing and new Employee IDs.	Existing records updated; new records inserted.	Correct behavior observed. Mix of "Updated" and "Inserted."	Pass
FT-07	Pie Chart Generation	Create a Pie Chart grouped by Department.	Chart displays correct proportions for each department.	Accurate percentages and counts displayed.	Pass
FT-08	Bar Chart Generation	Create a Bar Chart grouped by Location.	Chart displays correct counts for each location.	All bars reflect accurate data.	Pass
FT-09	Dashboard Widget Loading	Open the Employee Analytics Dashboard.	All 3 widgets load and display data correctly.	All widgets loaded successfully.	Pass
FT-10	ACL Admin Override	Attempt to create a dashboard without pa_admin role.	With Admin Override enabled, dashboard creation succeeds.	Dashboard created successfully by admin user.	Pass

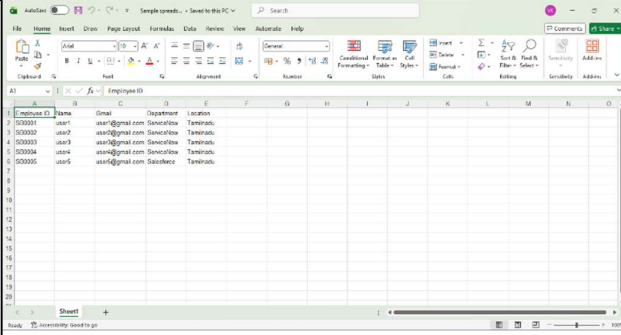
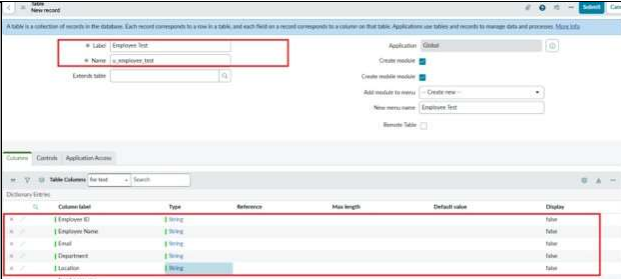
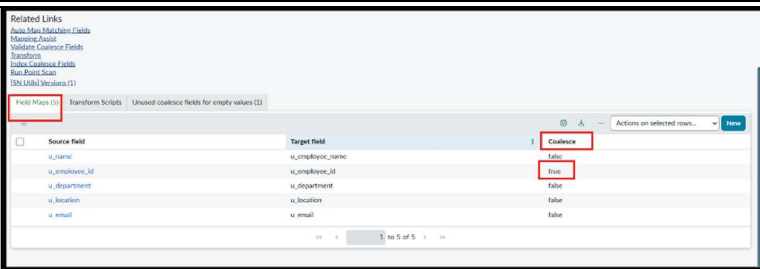
Performance Test Results

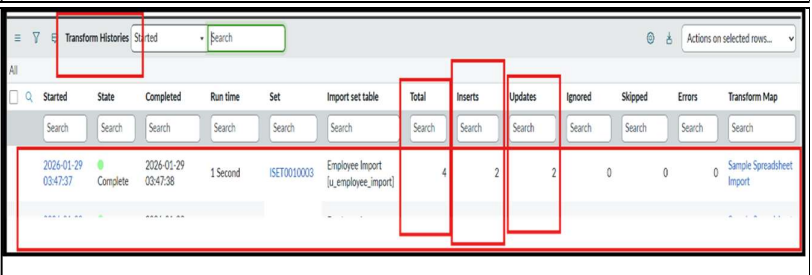
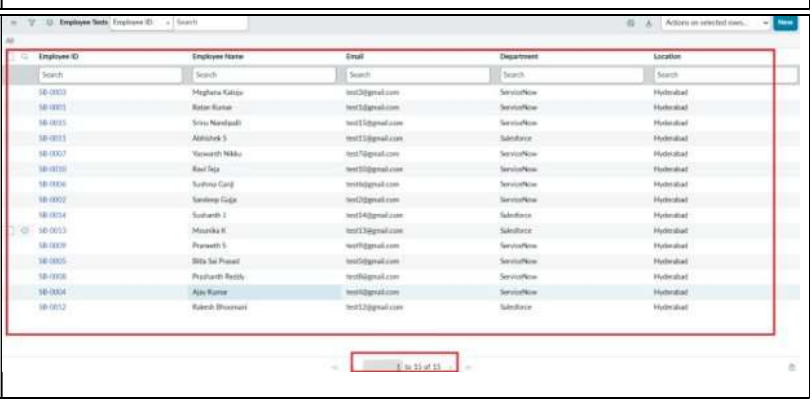
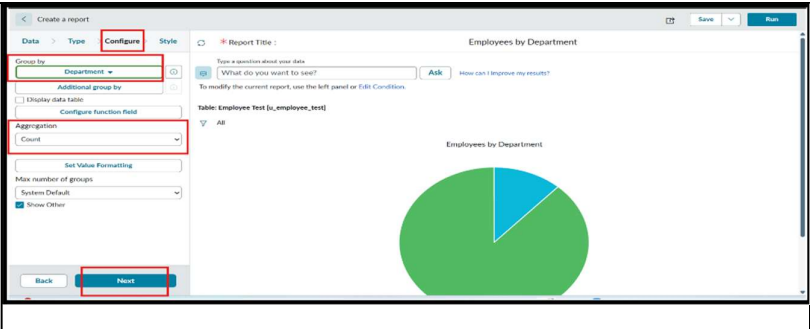
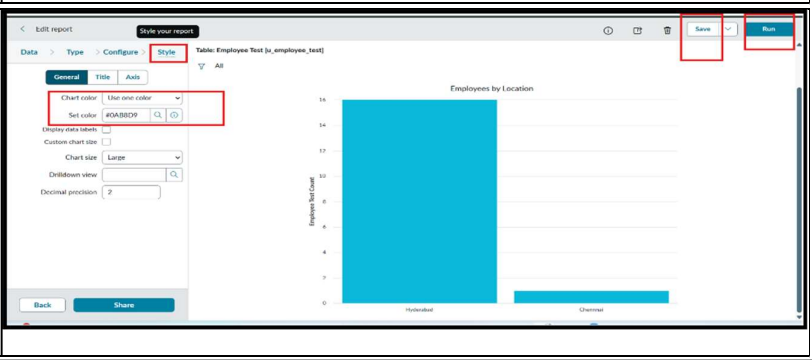
Test ID	Metric	Expected	Actual	Status
PT-01	Excel file upload time	< 5 seconds	~2 seconds	Pass
PT-02	Transform execution time (10 rows)	< 3 seconds	< 1 second	Pass
PT-03	Target table page load time	< 3 seconds	~1.5 seconds	Pass
PT-04	Dashboard rendering time (3 widgets)	< 5 seconds	~2 seconds	Pass

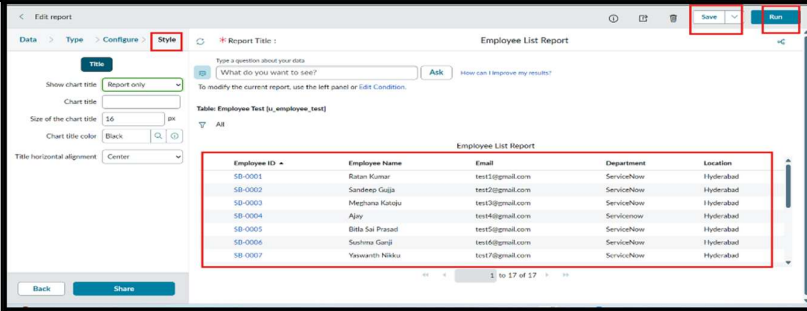
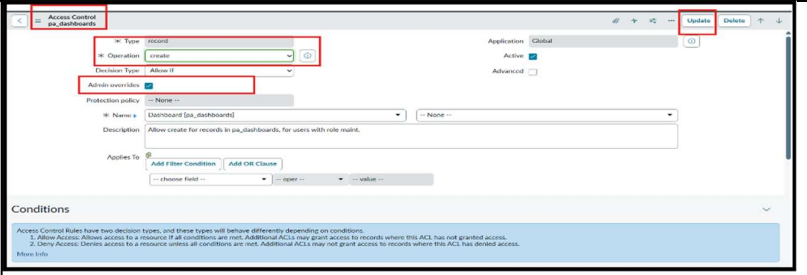
PT-05	Report generation time	< 3 seconds	~1 second	Pass
-------	------------------------	-------------	-----------	------

11. Screenshots or Demo

(Insert your actual project screenshots in the spaces below before final submission)

S.No	Screenshot Description	Placeholder
1	Sample Excel Spreadsheet with Employee Data	
2	Load Data Module — File Upload Screen	
3	Import Set Table — Staged Raw Data	
4	Custom Target Table (u_employee_test) — Table Schema	
5	Transform Map — Field Mappings with Coalesce Enabled	

6	Transform History — Showing "Inserted" Results	
7	Transform History — Showing "Updated" Results (Coalesce Test)	
8	Target Table — Final Employee Records	
9	Pie Chart — Employees by Department	
10	Bar Chart — Employees by Location	

11	List View Report — All Employees	
12	ACL Configuration — Admin Overrides Enabled on pa_dashboards	
13	Employee Analytics Dashboard — All Widgets	

Demo Link:

https://drive.google.com/file/d/1IDFOAN8UYJ4onAUn1is6iVmaGm35GkKW/view?usp=drive_link

GitHub Link:

https://github.com/vairamkishore104533-ai/Import_Data_Using_Transform_Maps

12. Known Issues

Issue ID	Description	Severity	Impact	Workaround / Status
KI-01	Column Header Sensitivity: If the Excel column headers do not exactly match the expected naming convention (e.g., "Emp ID" instead of "employee_id"), the Mapping Assist may fail to auto-detect the correct field mappings.	Medium	Requires manual field mapping intervention.	Ensure consistent column naming in all Excel files before upload. Provide a standardized template to all data providers.
KI-02	Dashboard ACL Restriction: By default, the pa_dashboards ACL does not allow users with only the admin role to create dashboards. This blocks the dashboard creation step unless the ACL is explicitly modified.	High	Completely blocks dashboard functionality until resolved.	Resolved — Enabled "Admin Overrides" on the pa_dashboards ACL rule.
KI-03	Import Set Table Cleanup: The staging table (u_employee_import) retains all previously uploaded data even after successful transformation. Over time, this can lead to unnecessary storage consumption.	Low	No functional impact, but increases database size.	Periodically clean up old Import Set records via System Import Sets > Administration > Cleanup.
KI-04	Single Coalesce Field Limitation: The current configuration uses only the Employee ID as the Coalesce field. If two different employees are accidentally assigned the same Employee ID in the source spreadsheet, the system will incorrectly merge their records.	Medium	Potential data corruption if source data quality is poor.	Implement a data validation script (Business Rule) on the target table to check for uniqueness before committing.

KI-05	Browser Compatibility: The ServiceNow Next Experience UI may render dashboard widgets differently across browsers. Minor visual inconsistencies have been observed in Safari and older versions of Firefox.	Low	Cosmetic differences only; no functional impact.	Use Google Chrome (latest version) for the most consistent experience.
-------	--	-----	--	--

13. Future Enhancements

The following future enhancements are proposed to extend the functionality, scalability, and automation capabilities of this project:

1. Scheduled Automated Imports via IntegrationHub Replace the manual file upload process with a fully automated integration using ServiceNow IntegrationHub or a Scheduled Import. This would allow the system to automatically pull employee data from an external source (e.g., an SFTP server, SharePoint folder, or cloud storage bucket) at predefined intervals (daily, weekly, monthly) without any manual intervention.

2. REST API Integration with External HR Systems Develop a custom Scripted REST API endpoint within ServiceNow that can receive employee data payloads from external HR management systems (e.g., Workday, SAP SuccessFactors, BambooHR) via secure REST API calls. This would enable real-time, event-driven data synchronization instead of batch-based file uploads.

3. Email Notifications on Import Completion Configure ServiceNow Email Notifications (or Event-driven alerts) to automatically send an email summary to the HR Manager after every successful data import. The email would include key metrics such as the number of records inserted, updated, and ignored, along with a direct link to the Employee Analytics Dashboard.

4. Advanced Data Validation with Business Rules Implement server-side Business Rules on the target table to enforce advanced data validation logic before records are committed. For example: validating email format (must contain "@" and a valid domain), checking for mandatory fields (no blank Employee IDs), and enforcing referential integrity (Department must exist in a master Department table).

5. Historical Trend Analytics with Performance Analytics (PA) Enable ServiceNow Performance Analytics (PA) indicators and breakdowns on the employee data to track historical trends over time. This would allow management to answer questions like "How has the IT department headcount changed over the past 12 months?" using time-series charts, trendlines, and scorecard widgets.

6. Role-Based Dashboard Access Create multiple dashboard views tailored to different user roles. For example, an HR Manager dashboard showing high-level department summaries, a Location Head dashboard filtered to show only employees in their specific location, and a System Admin dashboard showing Import Set health metrics and Transform History logs.

7. Mobile-Responsive Dashboard via ServiceNow Mobile App Extend the Employee Analytics Dashboard to be accessible via the ServiceNow Mobile App (available on iOS and Android). This would allow managers to view real-time employee analytics on their smartphones and tablets while on the go.

8. Data Export and Scheduled Report Distribution Configure the Report Builder to automatically generate and email PDF/Excel exports of the employee reports to executive stakeholders on a weekly or monthly schedule, eliminating the need for managers to manually log into the platform to access insights.

Prepared by:

Team ID: 6ab22227a614981d6d2c6451

Name	Role
VAIRAM KISHORE J	Project Lead & System Admin
SUDEEP KUMAR S	Data Engineer
VEERASEKARAN A	Testing & QA Lead
VELKUMAR S	Report & Dashboard Developer